AWT Assignments-4 Code

Krishna Patil
PRN: 125M1H025
Subject: AWT

Task 1

Question 1

Problem Statement

React Router – Student Portal Navigation…

Code

Filename: StudentPortal.jsx

```
import {
  Routes,
  Route,
  Link,
  useParams,
  useNavigate,
} from "react-router-dom";

import "./StudentPortal.css";

const students = [
  {
    id: 1,
    name: "Rahul Sharma",
    course: "MCA",
    year: "Second Year",
    email: "rahul@example.com",
  },
  {
    id: 2,
    name: "Priya Patil",
    course: "MCA",
    year: "Second Year",
    email: "priya@example.com",
  },
  {
    id: 3,
    name: "Amit Kulkarni",
    course: "MCA",
    year: "First Year",
    email: "amit@example.com",
  },
  {
    id: 4,
    name: "Sneha Joshi",
    course: "MCA",
    year: "First Year",
    email: "sneha@example.com",
  },
];

/* Home */

function Home() {
  return (
    <div className="portal-card">
      <h2>
        Welcome to Student Academic Portal
      </h2>

      <p>
        Manage student information and view
        academic details using this portal.
      </p>
    </div>
  );
}

/* Students */

function Students() {
  return (
    <div className="portal-card">
```

```
      <h2>Students</h2>

      <div className="student-list">

        {students.map((student) => (
          <Link
            key={student.id}
            to={`/router/students/${student.id}`}
            className="student-link"
          >
            <strong>
              {student.name}
            </strong>

            <span>
              {student.course}
            </span>
          </Link>
        ))}

      </div>

    </div>
  );
}

/* Student Details */

function StudentDetails() {
  const { id } = useParams();

  const navigate = useNavigate();

  const student = students.find(
    (student) => student.id === Number(id)
  );

  if (!student) {
    return (
      <div className="portal-card">

        <h2>Student Not Found</h2>

        <p>
          The requested student does not exist.
        </p>

        <button
          className="portal-button"
          onClick={() => navigate("/router/students")}
        >
          Back to Students
        </button>

      </div>
    );
  }

  return (
    <div className="portal-card">

      <h2>Student Details</h2>

      <div className="student-details">

        <p>
          <strong>ID:</strong>{" "}
          {student.id}
        </p>

        <p>
          <strong>Name:</strong>{" "}
          {student.name}
        </p>

        <p>
          <strong>Course:</strong>{" "}
          {student.course}
        </p>

        <p>
          <strong>Year:</strong>{" "}
          {student.year}
        </p>

        <p>
          <strong>Email:</strong>{" "}
          {student.email}
        </p>

      </div>

      <button
        className="portal-button"
        onClick={() => navigate("/router/students")}
      >
        Back to Students
```

```
      </button>

    </div>
  );
}

/* About */

function About() {
  return (
    <div className="portal-card">

      <h2>About the Portal</h2>

      <p>
        The Student Academic Portal is a React
        application demonstrating navigation
        and dynamic routes using React Router.
      </p>

    </div>
  );
}

/* 404 */

function NotFound() {
  return (
    <div className="portal-card not-found">

      <h2>404 - Page Not Found</h2>

      <p>
        Sorry, the page you are looking for
        does not exist.
      </p>

      <Link
        to="/router"
        className="portal-button"
      >
        Go to Home
      </Link>

    </div>
  );
}

/* Student Portal */

function StudentPortal() {
  return (
    <div className="portal">

      <div className="portal-header">

        <h1>
          Student Academic Portal
        </h1>

        <nav>

          <Link to="/router">
            Home
          </Link>

          <Link to="/router/students">
            Students
          </Link>

          <Link to="/router/about">
            About
          </Link>

        </nav>

      </div>

      <div className="portal-content">

        <Routes>

          <Route
            path="/"
            element={<Home />}
          />

          <Route
            path="/students"
            element={<Students />}
          />

          <Route
            path="/students/:id"
            element={<StudentDetails />}
          />

          <Route
```

```
            path="/about"
            element={<About />}
          />

          <Route
            path="*"
            element={<NotFound />}
          />

        </Routes>

      </div>

    </div>
  );
}

export default StudentPortal;
```

Filename: StudentPortal.css

```
.portal {
  min-height: 500px;
}

.portal-header {
  background: #1e293b;
  color: white;
  padding: 25px;
  border-radius: 10px;
}

.portal-header h1 {
  margin-bottom: 18px;
}

.portal-header nav {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
}

.portal-header nav a {
  color: white;
  text-decoration: none;
  font-weight: 600;
}

.portal-header nav a:hover {
  text-decoration: underline;
}

.portal-content {
  margin-top: 25px;
}

.portal-card {
  background: white;
  border: 1px solid #ddd;
  border-radius: 10px;
  padding: 25px;
  box-shadow: 0 3px 12px rgba(0, 0, 0, 0.06);
}

.portal-card h2 {
  margin-bottom: 15px;
}

.student-list {
  display: grid;
  gap: 12px;
  margin-top: 20px;
}

.student-link {
  display: flex;
  justify-content: space-between;
  padding: 15px;
  background: #f1f5f9;
  border-radius: 7px;
  text-decoration: none;
  color: #222;
}

.student-link:hover {
  background: #e2e8f0;
}

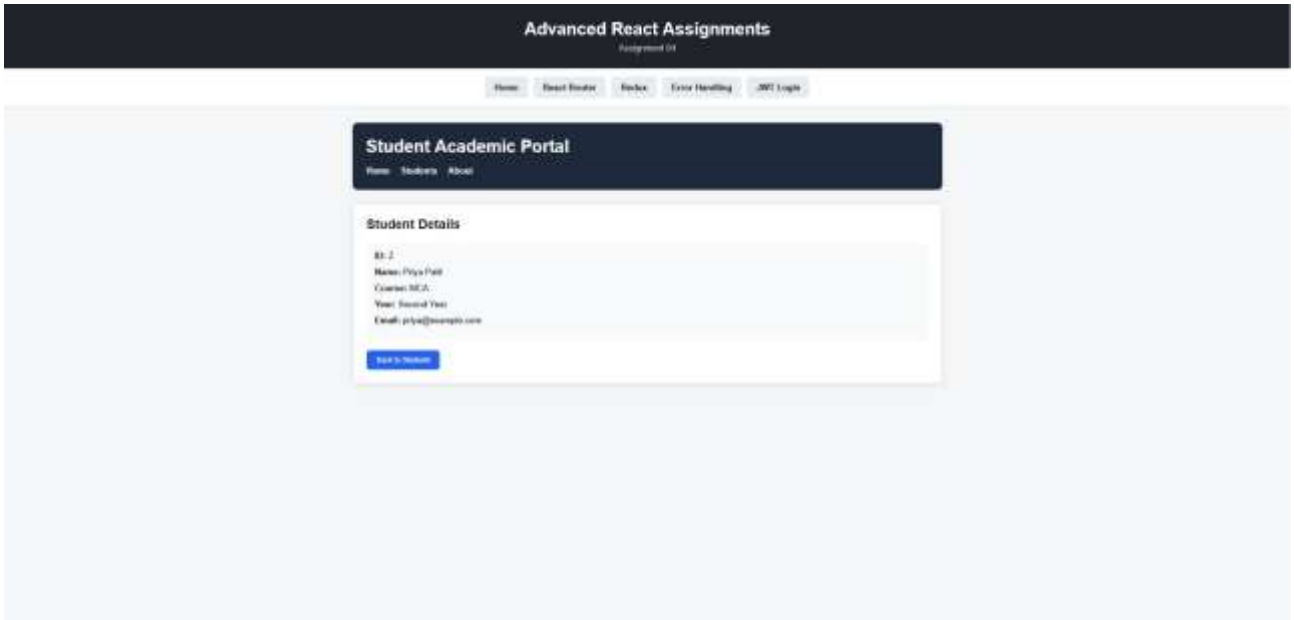
.student-details {
  margin: 20px 0;
  padding: 15px;
  background: #f8fafc;
  border-radius: 7px;
}

.student-details p {
  margin-bottom: 10px;
}
```

```
.portal-button {
  display: inline-block;
  border: none;
  background: #2563eb;
  color: white;
  padding: 10px 18px;
  border-radius: 6px;
  text-decoration: none;
  cursor: pointer;
}

.portal-button:hover {
  background: #1d4ed8;
}

.not-found {
  text-align: center;
}}
```



Task 2

Question 1

Problem Statement

React Redux – Expense Management…

Code

Filename: ExpenseApp.jsx

```
import { useState } from "react";

import {
  configureStore,
} from "@reduxjs/toolkit";

import {
  Provider,
  useDispatch,
  useSelector,
} from "react-redux";

import expenseReducer, {
  addExpense,
  deleteExpense,
} from "./expenseSlice";

import "./ExpenseApp.css";

/* Redux Store */

const store = configureStore({
  reducer: {
    expenses: expenseReducer,
  },
});

/* Expense Manager */
```

```
function ExpenseManager() {

  const dispatch = useDispatch();

  const expenses = useSelector(
    (state) => state.expenses.expenses
  );

  const [description, setDescription] =
    useState("");

  const [category, setCategory] =
    useState("");

  const [amount, setAmount] =
    useState("");

  const handleSubmit = (event) => {

    event.preventDefault();

    if (
      !description ||
      !category ||
      !amount
    ) {
      alert("Please fill all fields.");
      return;
    }

    dispatch(
      addExpense({
        description,
        category,
        amount,
      })
    );

    setDescription("");
    setCategory("");
    setAmount("");
  };

  const totalExpense =
    expenses.reduce(
      (total, expense) =>
        total + expense.amount,
      0
    );

  return (
    <div className="expense-container">

      <div className="expense-header">

        <h2>
          Expense Management
        </h2>

        <p>
          Manage expenses using React Redux.
        </p>

      </div>

      <form
        className="expense-form"
        onSubmit={handleSubmit}
      >

        <input
          type="text"
          placeholder="Description"
          value={description}
          onChange={(e) =>
            setDescription(e.target.value)
          }
        />

        <select
          value={category}
          onChange={(e) =>
            setCategory(e.target.value)
          }
        >

          <option value="">
            Select Category
          </option>

          <option value="Food">
            Food
          </option>

          <option value="Travel">
            Travel
          </option>
```

```
          <option value="Education">
            Education
          </option>

          <option value="Shopping">
            Shopping
          </option>

          <option value="Other">
            Other
          </option>

        </select>

        <input
          type="number"
          placeholder="Amount"
          value={amount}
          onChange={(e) =>
            setAmount(e.target.value)
          }
        />

        <button type="submit">
          Add Expense
        </button>

      </form>

      <div className="total-expense">
        Total Expense: ₹
        {totalExpense.toFixed(2)}
      </div>

      <div className="expense-list">

        {expenses.length === 0 ? (

          <p className="empty-message">
            No expenses added yet.
          </p>

        ) : (

          expenses.map((expense) => (

            <div
              className="expense-item"
              key={expense.id}
            >

              <div>

                <h3>
                  {expense.description}
                </h3>

                <p>
                  {expense.category}
                </p>

              </div>

              <div className="expense-right">

                <strong>
                  ₹{expense.amount.toFixed(2)}
                </strong>

                <button
                  className="delete-button"
                  onClick={() =>
                    dispatch(
                      deleteExpense(expense.id)
                    )
                  }
                >
                  Delete
                </button>

              </div>

            </div>

          ))

        )}

      </div>

    </div>
  );
}

/* Redux Provider */

function ExpenseApp() {
  return (
```

```
    <Provider store={store}>
      <ExpenseManager />
    </Provider>
  );
}

export default ExpenseApp;
```

Filename: ExpenseApp.css

```
.expense-container {
  max-width: 900px;
  margin: auto;
}

.expense-header {
  margin-bottom: 20px;
}

.expense-header h2 {
  margin-bottom: 6px;
}

.expense-header p {
  color: #666;
}

.expense-form {
  background: white;
  padding: 20px;
  border-radius: 10px;
  display: grid;
  grid-template-columns: 1.5fr 1fr 1fr auto;
  gap: 10px;
  border: 1px solid #ddd;
}

.expense-form input,
.expense-form select {
  padding: 11px;
  border: 1px solid #ccc;
  border-radius: 6px;
}

.expense-form button {
  background: #2563eb;
  color: white;
  border: none;
  padding: 10px 16px;
  border-radius: 6px;
  cursor: pointer;
}

.expense-form button:hover {
  background: #1d4ed8;
}

.total-expense {
  margin: 20px 0;
  padding: 18px;
  background: #eff6ff;
  border: 1px solid #bfdbfe;
  border-radius: 8px;
  font-size: 20px;
  font-weight: bold;
  color: #1d4ed8;
}

.expense-list {
  display: grid;
  gap: 12px;
}

.expense-item {
  background: white;
  padding: 18px;
  border-radius: 8px;
  border: 1px solid #ddd;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.expense-item h3 {
  margin-bottom: 5px;
}

.expense-item p {
  color: #777;
}

.expense-right {
  display: flex;
  align-items: center;
  gap: 15px;
}

.delete-button {
```

```
    background: #dc2626;
    color: white;
    border: none;
    padding: 8px 13px;
    border-radius: 5px;
    cursor: pointer;
  }

  .empty-message {
    text-align: center;
    color: #777;
    padding: 30px;
  }

  @media (max-width: 750px) {
    .expense-form {
      grid-template-columns: 1fr;
    }

    .expense-item {
      flex-direction: column;
      align-items: flex-start;
      gap: 15px;
    }
}
```

Filename: expenseSlice.js

```
  import { createSlice } from "@reduxjs/toolkit";

  const initialState = {
    expenses: [],
  };

  const expenseSlice = createSlice({
    name: "expenses",

    initialState,

    reducers: {

      addExpense: (state, action) => {
        state.expenses.push({
          id: Date.now(),

          description:
            action.payload.description,

          category:
            action.payload.category,

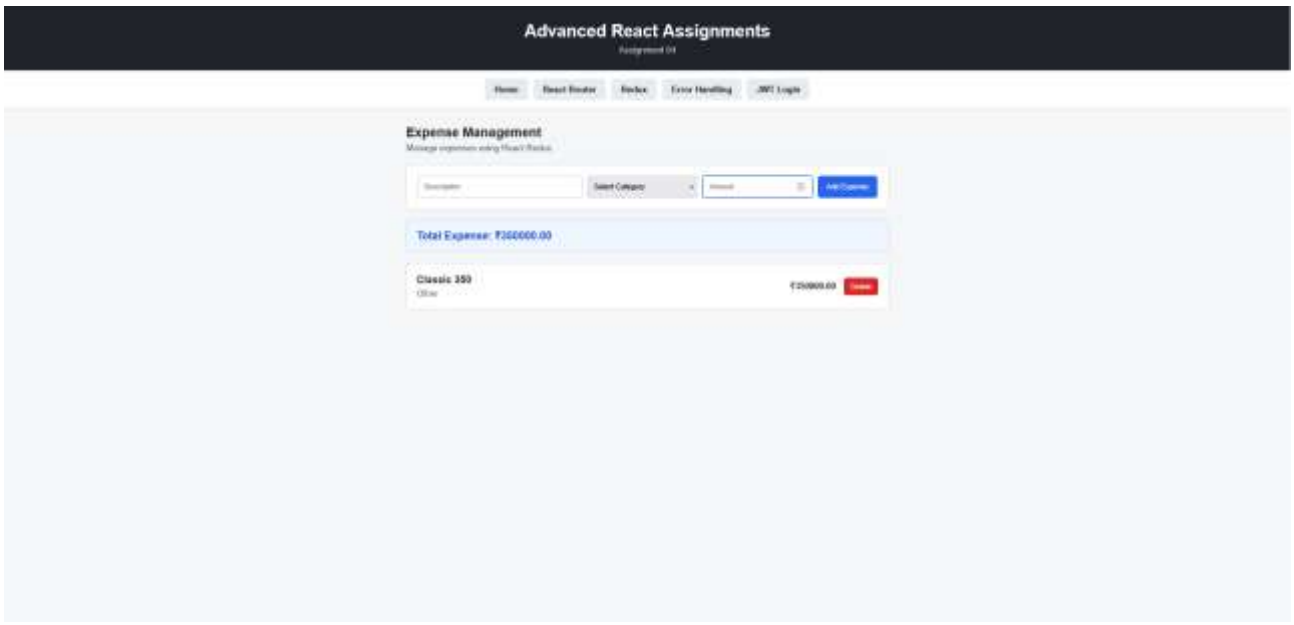
          amount:
            Number(action.payload.amount),
        });
      },

      deleteExpense: (state, action) => {
        state.expenses =
          state.expenses.filter(
            (expense) =>
              expense.id !== action.payload
          );
      },

    },
  });

  export const {
    addExpense,
    deleteExpense,
  } = expenseSlice.actions;

  export default expenseSlice.reducer;
```



Task 3

Question 1

Problem Statement

React Error Handling – Online Course Portal...

Code

Filename: CoursePortal.jsx

```
import React from "react";
import "./CoursePortal.css";

/* Error Boundary */

class ErrorBoundary extends React.Component {

  constructor(props) {
    super(props);

    this.state = {
      hasError: false,
      error: null,
    };
  }

  static getDerivedStateFromError(error) {

    return {
      hasError: true,
      error: error,
    };
  }

  componentDidCatch(error, errorInfo) {

    console.error(
      "Course Portal Error:",
      error,
      errorInfo
    );
  }

  handleTryAgain = () => {

    this.setState({
      hasError: false,
      error: null,
    });

  };

  render() {

    if (this.state.hasError) {

      return (
        <div className="error-container">
```

```
          <h2>
            Something went wrong
          </h2>

          <p>
            We could not load this course
            information. Please try again.
          </p>

          <button
            className="course-button"
            onClick={this.handleTryAgain}
          >
            Try Again
          </button>

        </div>
      );
    }

    return this.props.children;
  }
}

/* Course Data */

const courses = [
  {
    id: 1,
    name: "React Fundamentals",
    instructor: "John Smith",
    duration: "6 Weeks",
  },

  {
    id: 2,
    name: "Advanced JavaScript",
    instructor: "Sarah Johnson",
    duration: "8 Weeks",
  },

  {
    id: 3,
    name: "Node.js Development",
    instructor: "David Wilson",
    duration: "7 Weeks",
  },
];

/* Course Card */

function CourseCard({
  course,
  onSelect,
}) {

  return (
    <div className="course-card">

      <h3>
        {course.name}
      </h3>

      <p>
        <strong>
          Instructor:
        </strong>{" "}
        {course.instructor}
      </p>

      <p>
        <strong>
          Duration:
        </strong>{" "}
        {course.duration}
      </p>

      <button
        className="course-button"
        onClick={() =>
          onSelect(course)
        }
      >
        View Details
      </button>

    </div>
  );
}

/* Course Details */

function CourseDetails({
  course,
}) {

  if (
    !course ||
```

```
    !course.name
  ) {

    throw new Error(
      "Invalid course data supplied."
    );
  }

  return (
    <div className="course-details">

      <h2>
        {course.name}
      </h2>

      <p>
        <strong>
          Instructor:
        </strong>{" "}
        {course.instructor}
      </p>

      <p>
        <strong>
          Duration:
        </strong>{" "}
        {course.duration}
      </p>

      <p>
        This course provides practical
        knowledge and hands-on experience.
      </p>

    </div>
  );
}

/* Course List */

function CourseList({
  onSelect,
}) {

  return (
    <div className="course-list">

      {courses.map((course) => (

        <CourseCard
          key={course.id}
          course={course}
          onSelect={onSelect}
        />

      ))}

    </div>
  );
}

/* Main Course Portal */

function CoursePortal() {

  const [
    selectedCourse,
    setSelectedCourse,
  ] = React.useState(null);

  return (
    <div className="course-portal">

      <div className="course-header">

        <h1>
          Online Course Portal
        </h1>

        <p>
          Learn new skills through
          online courses.
        </p>

      </div>

      <CourseList
        onSelect={setSelectedCourse}
      />

      {selectedCourse && (

        <ErrorBoundary>

          <CourseDetails
            course={selectedCourse}
          />
```

```
        </ErrorBoundary>

      )}

    </div>
  );
}

export default CoursePortal;
```

Filename: CoursePortal.css

```
.course-portal {
  max-width: 1000px;
  margin: auto;
}

.course-header {
  background: #1e293b;
  color: white;
  padding: 25px;
  border-radius: 10px;
  margin-bottom: 25px;
}

.course-header h1 {
  margin-bottom: 8px;
}

.course-list {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 18px;
}

.course-card {
  background: white;
  border: 1px solid #ddd;
  border-radius: 9px;
  padding: 20px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.05);
}

.course-card h3 {
  margin-bottom: 12px;
}

.course-card p {
  margin-bottom: 8px;
  color: #555;
}

.course-button {
  margin-top: 12px;
  background: #2563eb;
  color: white;
  border: none;
  padding: 10px 16px;
  border-radius: 6px;
  cursor: pointer;
}

.course-button:hover {
  background: #1d4ed8;
}

.course-details {
  margin-top: 25px;
  padding: 25px;
  background: #eff6ff;
  border: 1px solid #bfdbfe;
  border-radius: 10px;
}

.course-details h2 {
  margin-bottom: 15px;
}

.course-details p {
  margin-bottom: 10px;
}

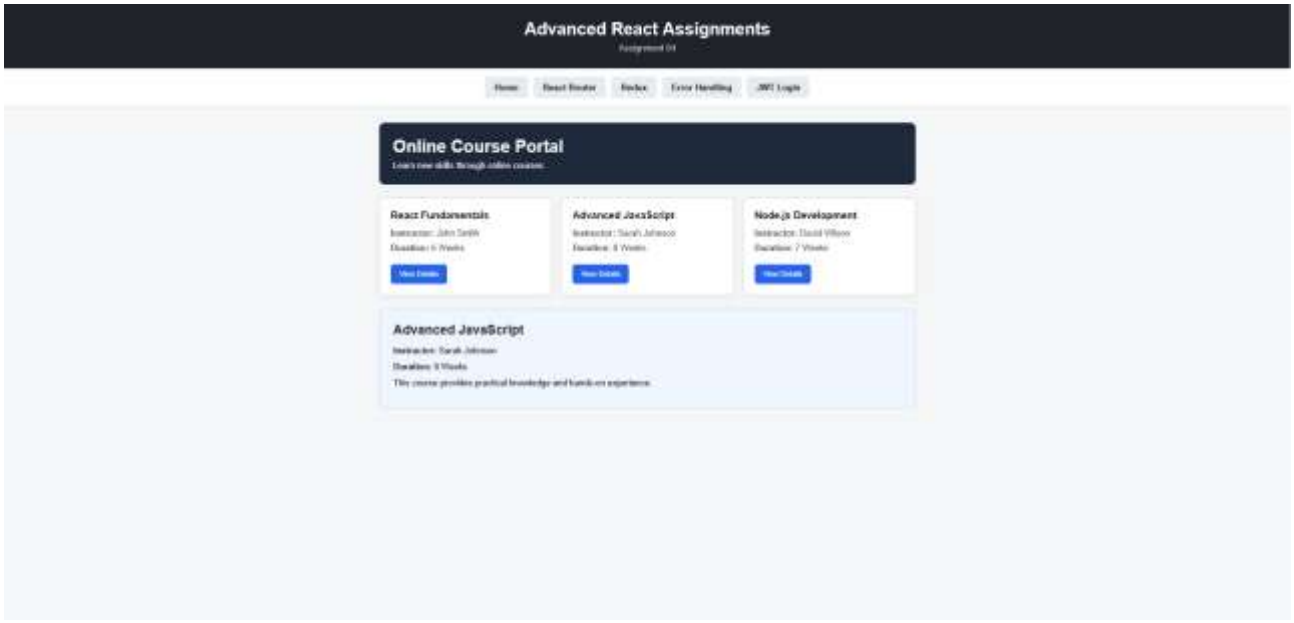
.error-container {
  margin-top: 25px;
  padding: 30px;
  background: #fef2f2;
  border: 1px solid #fecaca;
  border-radius: 10px;
  text-align: center;
}

.error-container h2 {
  color: #dc2626;
  margin-bottom: 10px;
}

.error-container p {
  margin-bottom: 15px;
```

```
  }

  @media (max-width: 750px) {
    .course-list {
      grid-template-columns: 1fr;
    }
}
```



Task 4

Question 1

Problem Statement

JWT – Secure Student Login System…

Code

Filename: LoginApp.jsx

```
import { useState } from "react";
import "./LoginApp.css";

const API_URL = "http://localhost:5000";

function LoginApp() {

  const [username, setUsername] =
    useState("");

  const [password, setPassword] =
    useState("");

  const [token, setToken] =
    useState(
      localStorage.getItem("token")
    );

  const [user, setUser] =
    useState(null);

  const [message, setMessage] =
    useState("");

  const handleLogin = async (event) => {

    event.preventDefault();

    setMessage("");

    try {

      const response =
        await fetch(
          `${API_URL}/api/login`,
          {
            method: "POST",

            headers: {
              "Content-Type":
```

```
              "application/json",
          },

          body: JSON.stringify({
            username,
            password,
          }),
        }
      );

    const data =
      await response.json();

    if (!response.ok) {

      setMessage(
        data.message
      );

      return;
    }

    localStorage.setItem(
      "token",
      data.token
    );

    setToken(data.token);

    setMessage(
      data.message
    );

    fetchDashboard(
      data.token
    );

  } catch {

    setMessage(
      "Unable to connect to the authentication server."
    );
  }
};

const fetchDashboard =
  async (authToken) => {

    try {

      const response =
        await fetch(
          `${API_URL}/api/dashboard`,
          {
            headers: {
              Authorization:
                `Bearer ${authToken}`,
            },
          }
        );

      const data =
        await response.json();

      if (!response.ok) {

        localStorage.removeItem(
          "token"
        );

        setToken(null);

        setMessage(
          data.message
        );

        return;
      }

      setUser(
        data.user
      );

    } catch  {

      setMessage(
        "Unable to access the dashboard."
      );
    }
  };

const handleLogout = () => {

  localStorage.removeItem(
    "token"
  );

  setToken(null);
```

```
  setUser(null);

  setUsername("");
  setPassword("");

  setMessage(
    "You have been logged out."
  );
};

/* Dashboard */

if (token && user) {

  return (
    <div className="jwt-container">

      <div className="dashboard-card">

        <h2>
          Student Dashboard
        </h2>

        <p className="success-message">
          Authentication successful!
        </p>

        <div className="user-info">

          <p>
            <strong>
              Username:
            </strong>{" "}
            {user.username}
          </p>

          <p>
            <strong>
              Email:
            </strong>{" "}
            {user.email}
          </p>

        </div>

        <button
          className="logout-button"
          onClick={handleLogout}
        >
          Logout
        </button>

      </div>

    </div>
  );
}

/* Login */

return (
  <div className="jwt-container">

    <div className="login-card">

      <h2>
        Student Login
      </h2>

      <p>
        Login to access the student
        dashboard.
      </p>

      <form
        onSubmit={handleLogin}
      >

        <label>
          Username / Email
        </label>

        <input
          type="text"
          value={username}
          onChange={(e) =>
            setUsername(
              e.target.value
            )
          }
          placeholder="student"
        />

        <label>
          Password
        </label>

        <input
```

```
            type="password"
            value={password}
            onChange={(e) =>
              setPassword(
                e.target.value
              )
            }
            placeholder="Enter password"
          />

          <button
            type="submit"
            className="login-button"
          >
            Login
          </button>

        </form>

        {message && (

          <div className="login-message">
            {message}
          </div>

        )}

        <div className="login-help">

          <strong>
            Demo Credentials
          </strong>

          <p>
            Username: student
          </p>

          <p>
            Password: 123456
          </p>

        </div>

      </div>

    </div>
  );
}

export default LoginApp;
```

Filename: LoginApp.css

```
.jwt-container {
  max-width: 600px;
  margin: auto;
}

.login-card,
.dashboard-card {
  background: white;
  padding: 30px;
  border-radius: 10px;
  border: 1px solid #ddd;
  box-shadow: 0 3px 12px rgba(0, 0, 0, 0.06);
}

.login-card h2,
.dashboard-card h2 {
  margin-bottom: 8px;
}

.login-card > p {
  color: #666;
  margin-bottom: 25px;
}

.login-card form {
  display: flex;
  flex-direction: column;
}

.login-card label {
  margin-bottom: 6px;
  font-weight: 600;
}

.login-card input {
  padding: 11px;
  margin-bottom: 17px;
  border: 1px solid #ccc;
  border-radius: 6px;
}

.login-button,
.logout-button {
  border: none;
  padding: 11px;
```

```
    border-radius: 6px;
    cursor: pointer;
    color: white;
    font-weight: 600;
  }

  .login-button {
    background: #2563eb;
  }

  .login-button:hover {
    background: #1d4ed8;
  }

  .logout-button {
    background: #dc2626;
    margin-top: 20px;
  }

  .logout-button:hover {
    background: #b91c1c;
  }

  .login-message {
    margin-top: 15px;
    padding: 12px;
    background: #fef2f2;
    color: #b91c1c;
    border-radius: 6px;
  }

  .login-help {
    margin-top: 20px;
    padding: 15px;
    background: #f8fafc;
    border-radius: 6px;
    color: #555;
  }

  .login-help p {
    margin-top: 5px;
  }

  .success-message {
    padding: 12px;
    background: #f0fdf4;
    color: #15803d;
    border-radius: 6px;
    margin: 15px 0;
  }

  .user-info {
    background: #f8fafc;
    padding: 15px;
    border-radius: 7px;
  }

  .user-info p {
    margin-bottom: 8px;
}
```

Filename: server.cjs

```
const express = require("express");
const jwt = require("jsonwebtoken");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

const PORT = 5000;

const JWT_SECRET =
  "student_portal_secret";

/* Demo Student */

const student = {
  username: "student",
  email: "student@example.com",
  password: "123456",
};

/* Login */

app.post(
  "/api/login",
  (req, res) => {

    const {
      username,
      password,
    } = req.body;
```

```
    if (
      username !== student.username &&
      username !== student.email
    ) {

      return res.status(401).json({
        message:
          "Invalid username or email.",
      });
    }

    if (
      password !== student.password
    ) {

      return res.status(401).json({
        message:
          "Invalid password.",
      });
    }

    const token =
      jwt.sign(
        {
          username:
            student.username,

          email:
            student.email,
        },

        JWT_SECRET,

        {
          expiresIn: "1h",
        }
      );

    res.json({
      message:
        "Login successful",

      token,
    });
  }
);

/* Authentication Middleware */

function authenticateToken(
  req,
  res,
  next
) {

  const authHeader =
    req.headers.authorization;

  if (!authHeader) {

    return res.status(401).json({
      message:
        "Authentication token missing.",
    });
  }

  const token =
    authHeader.split(" ")[1];

  if (!token) {

    return res.status(401).json({
      message:
        "Invalid authentication token.",
    });
  }

  jwt.verify(
    token,
    JWT_SECRET,
    (error, user) => {

      if (error) {

        return res.status(403).json({
          message:
            "Token expired or invalid.",
        });
      }

      req.user = user;

      next();
    }
  );
}

/* Protected Dashboard */
```

```
app.get(
  "/api/dashboard",
  authenticateToken,
  (req, res) => {

    res.json({
      message:
        "Welcome to the Student Dashboard!",

      user:
        req.user,
    });
  }
);

/* Start Server */

app.listen(
  PORT,
  () => {

    console.log(
      `Server running at http://localhost:${PORT}`
    );
  }
);
```

